

Lezione — FI2 01: Dai linguaggi alle infrastrutture software

Corso: Fondamenti di Informatica T-2 (12 CFU · S1) **Materiale:** [materiali/slide/01-x1-Intro.pdf](#) — E. Denti, *Dai Linguaggi alle Infrastrutture Software*, a.a. 2023/24, 101 slide **Prerequisiti:** nessuno nel corso. Presuppone il modello di compilazione del C (compilatore → linker → eseguibile), che le slide richiamano senza rispiegarlo.

Annata: slide 2023/24, l'edizione d'iscrizione ([fonti.md](#)). Le classifiche di popolarità dei linguaggi (sl. 54-67) e le versioni citate (.NET 8.0, Java SE 21) sono fotografie di quell'anno: non sono materia d'esame e invecchiano.

Cosa non è coperto dal testo estratto: alcune slide sono solo immagini — l'articolo su Google (sl. 16), le schermate SVN/Git (sl. 10-12), i diagrammi UML (sl. 71-72), le rassegne stampa «Gli introvabili» (sl. 36-45). La lezione non ne ricostruisce il contenuto.

Obiettivo

Saper spiegare, con le parole del docente, perché il corso non insegna a «programmare» ma a progettare sistemi software, e perché questo porta con sé tre cose che ritroverai in ogni compito d'esame: il collaudo automatico con JUnit, la separazione fra progetto e codice, e un eseguibile (il JAR) che non gira senza la sua infrastruttura.

1. Da «programmare» a «progettare»: la crisi del software

Il punto di partenza del docente è una parola. «Programmare» richiama un'attività in cui le fasi creative, cioè analisi e progetto, sono già avvenute, ed è «tipico della costruzione di algoritmi di medio/piccola dimensione, normalmente già noti e descritti in letteratura» [fonte: 01-Intro, sl. 2]. Negli anni '80 questa visione va in crisi: la **crisi del software** è l'«insufficienza di metodologie e strumenti nati per una visione “algoritmica” dell'informatica (programmazione in-the-small) rispetto a progettazione, sviluppo e manutenzione di sistemi software complessi (progettazione in the large)» [fonte: 01-Intro, sl. 2].

Cosa c'è dietro. La coppia *in-the-small* / *in-the-large* non misura le righe di codice: distingue due problemi diversi. Nel piccolo il problema è l'algoritmo; nel grande è coordinare molte parti, molte persone e molte versioni nel tempo. Da qui il nuovo obiettivo: «saper progettare e costruire sistemi software con il rigore e le garanzie che si richiedono a tutti gli altri prodotti dell'ingegneria», cioè qualità e affidabilità, uscendo dall'artigianato e concependo il software «come un prodotto industriale» [fonte: 01-Intro, sl. 3].

La visione. Il docente scompone il «prodotto industriale» in cinque requisiti [fonte: 01-Intro, sl. 4]: un processo di produzione strutturato, il versioning, il **collaudo sistematico come parte integrante del processo**, l'automazione dei passi chiave e la

cura degli aspetti legali (privacy by design and by default, GDPR). Le sezioni che seguono sono queste cinque voci sviluppate una per una. Ogni pezzo di Java che vedrai nel corso risponde a una di esse.

Il ragionamento che regge tutta la presentazione sta nel **tema della scalabilità**: l'essere umano tende a pensare che ciò che ha funzionato su 3 elementi funzioni anche su 3000, «ma nei fatti gli strumenti (fisici e mentali) quasi mai scalano all'aumentare della complessità!» [fonte: 01-Intro, sl. 48]. L'immagine del docente è questa: «per avvitare cinque viti basta un cacciavite.. ma per la Tour Eiffel?»

2. Il controllo di versione

Cosa fa. Un sistema di versioning permette di tenere traccia di ogni versione di ogni file, tornare indietro «senza mai perdere niente anche in caso di sbagli», gestire diramazioni sperimentali (*branch*), riunire versioni diverse (*merge*), ottenere report sulle differenze, gestire le dipendenze e supportare il lavoro di squadra, «chi ha fatto cosa e quando», anche offline [fonte: 01-Intro, sl. 6].

Cosa c'è dietro. Il problema che risolve è precisato prima dello strumento: lo sviluppo «passa attraverso molti file», si fanno modifiche e ci si pente, più persone lavorano insieme, e farlo col copia-e-incolla significa che «non si sa più cosa c'è in ogni file» e «non si riescono a mantenere coerenti versioni successive» [fonte: 01-Intro, sl. 5]. L'idea base: i file locali «sono copiati, a richiesta, su un repository che traccia le versioni» [fonte: 01-Intro, sl. 7].

Qui c'è la prima distinzione da non collassare:

- in **SVN** il repository è **unico e centralizzato**;
- in **Git** «ogni pc ha il suo repository (distribuito), che solo a richiesta è copiato nel repository centrale remoto», ed è oggi il più diffuso [fonte: 01-Intro, sl. 7].

⚠ **Distinzione da tenere separata** (profilo/errori.md, trasversale n. 1). «Salvare nel repository» non è la stessa operazione nei due sistemi. Il caso limite che li separa è quello che hai vissuto stamattina: il commit di ~/cockpit era fatto in locale e il lavoro esisteva già come versione, ma su GitHub non c'era ancora nulla finché non è partito il push. In SVN quel momento intermedio non esiste, perché il commit va direttamente nel repository centrale. È questo che rende Git utilizzabile «anche offline».

La visione. Il versioning viene prima di qualsiasi riga di Java perché è ciò che rende possibile tutto il resto del processo. La Continuous Integration (§3) presuppone che «il codice è sotto controllo di versione» [fonte: 01-Intro, sl. 15]: senza una versione precedente da ripristinare, un test fallito non ha un rimedio.

3. Il collaudo: JUnit e la Continuous Integration

È il blocco più importante della presentazione per l'esame, perché la prova di FI2 si supera con codice che «deve compilare e superare almeno 2/3 dei test forniti» (fonti.md,

scheda 2024/25). I test del compito sono il collaudo di cui parla questa sezione.

Cosa fa. Il collaudo «non si improvvisa: va accuratamente progettato — non dopo: prima ancora di costruire il sistema»; è «parte integrante del TUO lavoro», va strumentato ed è espresso dal **piano di collaudo**, che «guida la validazione del sistema». Soprattutto: «non è fare “qualche stampa” (chi le legge? se sono centinaia?)» [fonte: 01-Intro, sl. 13]. Lo strumento per Java è **JUnit**, che «esegue in automatico i test progettati, riportandone i risultati in forma grafica» [fonte: 01-Intro, sl. 18].

Cosa c'è dietro — due distinzioni.

La prima riguarda i due approcci al collaudo [fonte: 01-Intro, sl. 14]:

- **black-box testing**: progettato e svolto «“ai morsetti” di un componente software, senza sapere com'è fatto dentro»; lo scopo è verificare «la correttezza del comportamento esterno, indipendentemente dalla realizzazione»;
- **white-box testing**: progettato «tenendo esplicitamente in conto com'è fatto dentro un componente»; lo scopo è verificare «la corretta implementazione delle varie funzioni, stressandole una ad una anche nei casi critici».

La seconda compare nella schermata di JUnit, e il docente la lascia in inglese [fonte: 01-Intro, sl. 18]:

«A **failure** is when one of your assertions fails. An **error** is when some other exception occurs, one you haven't tested for and didn't expect.»

△ **Distinzione da tenere separata** (trasversale n. 1). Failure ed error sono due esiti diversi di un test non superato. Il caso limite che li separa: un metodo che restituisce il valore sbagliato produce una *failure*, perché il test è arrivato all'asserzione e il confronto non torna. Lo stesso metodo che invece solleva un'eccezione imprevista prima di restituire qualsiasi cosa produce un *error*, perché l'asserzione non viene mai raggiunta. All'esame la distinzione ti dice dove cercare: con una failure la logica è sbagliata, con un error il codice si rompe prima.

△ **Fermarsi al primo indizio** (trasversale n. 2). Qualche test verde non chiude il compito. Il criterio è quantitativo, 2/3 dei test, e il collaudo del docente è progettato «anche nei casi critici». Un'esecuzione riuscita sul caso che hai in mente non spiega *tutti* i test.

La visione — Continuous Integration. Il collaudo si inserisce in una strategia di sviluppo [fonte: 01-Intro, sl. 15]: il codice è sotto controllo di versione; a ogni modifica si rieseguono i test e si ottengono report dettagliati; «solo se OK, la modifica è approvata; altrimenti, si corregge o si ripristina la versione precedente»; ogni tanto si rilascia una versione consistente (*release*). A sostegno del ciclo esistono i **build tools** come Gradle e Maven: in Gradle si specifica «con un apposito linguaggio, il grafo delle dipendenze fra più progetti + il relativo ciclo di sviluppo» [fonte: 01-Intro, sl. 17].

Le slide 19-27 sono una galleria di bug reali: un orario ferroviario che ignora la linea del cambio di data, una data 31/07/2011 con i campi invertiti sugli estratti conto «e nessuno se ne accorge per mesi», prezzi con la virgola e col punto nella stessa pagina. Non sono materia da ricordare, ma la loro funzione è precisa: sono tutti errori che un piano di collaudo sistematico avrebbe intercettato.

4. Collaudo contro progetto: quando il test non basta

Cosa mostra. Le slide 28–30 sono diverse dalla galleria precedente, e il titolo lo dice: *collaudo vs progetto*. Qui l'errore non è un bug di implementazione: sono «le assunzioni di base non rispettate a generare assurdità al momento dell'uso» [fonte: 01-Intro, sl. 28].

Cosa c'è dietro. Il caso «bolognese» del pianificatore del 2016 [fonte: 01-Intro, sl. 29]: per andare da Aldini a Carducci bastava restare sul 33, ma il software suggeriva di scendere a Porta S. Mamolo e aspettare il bus successivo. Il motivo è che chi l'aveva scritto aveva ipotizzato «che tutte le linee abbiano due capilinea». La linea circolare violava l'assunzione ed era stata modellata come due tronconi, andata e ritorno. Il software, «tarato (giustamente) sull'assunzione di base, vede un itinerario composto». L'altro caso è una compagnia aerea che non prevedeva passeggeri minorenni: il docente lo etichetta «**Dominio del problema mal analizzato**» [fonte: 01-Intro, sl. 28].

△ **Distinzione da tenere separata** (trasversale n. 1). Un bug di collaudo e un errore di progetto si somigliano all'uscita, perché in entrambi il software dà un risultato assurdo, ma hanno cause e rimedi diversi. Il caso limite è proprio il 33: il software faceva *esattamente* ciò per cui era progettato, quindi nessun test scritto sul modello a due capilinea sarebbe fallito. L'errore stava nel modello del dominio, a monte del codice.

La visione. Questa sezione prepara il §5: se l'errore può stare nel modello e non nel codice, allora il modello deve esistere come oggetto a sé, da poter esaminare.

5. Il progetto non è il codice

Cosa afferma. «Il progetto di un sistema software non è il codice — esattamente come il progetto di una casa non è la casa, il progetto di un ponte non è il ponte stesso!» [fonte: 01-Intro, sl. 68]. Il risultato dell'attività di progetto «dev'essere un artefatto esistente e documentato in quanto tale», espresso come nel resto dell'ingegneria con disegni e diagrammi che trasmettono «la struttura d'insieme» e «le parti coinvolte e le loro relazioni».

Cosa c'è dietro. Due affermazioni forti [fonte: 01-Intro, sl. 70]:

- l'attività di progetto «dev'essere **precedente e totalmente disaccoppiata** dall'implementazione»;
- «non si gestisce la complessità senza poter operare a un adeguato livello di astrazione — che non è il codice».

Da qui il ruolo di **UML** (Unified Modelling Language), «uno standard internazionale per esprimere modelli di sistemi (non necessariamente informatici)». Fra i suoi diagrammi, il **diagramma di struttura** «esprime la struttura di un sistema a oggetti» [fonte: 01-Intro, sl. 71]. La slide 72 ne mostra un pezzo «da un compito d'esame».

La visione. Nel compito d'esame il diagramma UML è il progetto che ti viene consegnato e il codice è l'implementazione che produci tu. Leggere il diagramma prima di scrivere non è una formalità: è la parte del lavoro che il docente considera ingegneria. La slide 46 lo formula come requisito professionale: «comprendere che il progetto non è il codice» e «comprendere che progettare non è implementare».

Il perché dei linguaggi a oggetti. Il C e i linguaggi classici «non erano pensati per supportare un'attività di progetto — il progetto “era” il codice (o al massimo un diagramma di flusso..)», né per un collaudo sistematico: i test «erano tipicamente mischiati nel codice» [fonte: 01-Intro, sl. 49]. Il linguaggio moderno non può limitarsi agli elementi linguistici: «deve fornire concetti, metafore e strumenti adatti a governare la complessità dei sistemi software» [fonte: 01-Intro, sl. 50]. I linguaggi a oggetti sono «uno dei maggiori casi di successo degli ultimi decenni», e oggi sono sempre più spesso **blended** con concetti dei linguaggi funzionali [fonte: 01-Intro, sl. 51].

Le idee prese dal mondo funzionale sono elencate alla slide 53: la distinzione variabili/valori (var vs val), «everything is an object», «everything is an expression», le strutture dati immodificabili («compute by synthesis»), le funzioni come *first-class entities* (chiusure, lambda, lazy evaluation) e uno stile più conciso. È una mappa di ciò che arriva più avanti nel corso: vedi *Connessioni*.

6. Dai linguaggi alle infrastrutture: cosa è un «eseguibile»

Questa è la seconda metà della presentazione e dà il titolo al modulo.

Cosa afferma. «Un buon linguaggio non basta»: intorno a esso serve un **ecosistema** pensato secondo una precisa visione infrastrutturale. Il controesempio del docente è il C++: «poter creare propri componenti e oggetti è importante, ma standardizzare il linguaggio non basta». Se ognuno scrive le proprie librerie «circa simili, ma non identiche» nascono «comunità incomunicabili» [fonte: 01-Intro, sl. 73]. Serve un'**infrastruttura standard**: componenti pronti presenti su ogni installazione, disponibile per più sistemi operativi e hardware, uno strato «che renda indipendente chi sta sopra dalla specifica configurazione hardware/software sottostante». L'obiettivo è «write once, run everywhere» [fonte: 01-Intro, sl. 74].

Cosa c'è dietro — il confronto fra i due modelli di compilazione.

Nel «**mondo C**» [fonte: 01-Intro, sl. 78 e 82]: il sorgente C passa dal compilatore, il linker lo collega **staticamente** alle librerie in formato oggetto (OBJ), e il risultato è un **eseguibile autocontenuto** (EXE) generato per quello specifico sistema operativo.

Nel «**nuovo mondo**» [fonte: 01-Intro, sl. 78 e 83]: il compilatore genera componenti «nel formato oggetto dell'infrastruttura (BYTECODE o MSIL)», non collega librerie staticamente, e «si caricano e collegano DINAMICAMENTE, a run-time, i componenti che servono». Il prodotto è un **archivio**, JAR in Java o Assembly in .NET, «“eseguibile” dall'infrastruttura». L'applicazione «non è più una entità monolitica, auto-contenuta, ma un cliente di servizi dell'infrastruttura».

In Java il sorgente è strutturato in componenti, «i più importanti dei quali si chiamano **classi**» (in Scala e Kotlin anche oggetti singleton). Ognuno è compilato in un formato

portabile e l'eseguibile può essere compilato su un sistema operativo e girare su un altro, «purché la macchina sia dotata della stessa infrastruttura». Per farlo serve uno **strato-ponte** che interpreti il formato portabile adattandolo alla macchina: «È l'unico strato dipendente dalla piattaforma» [fonte: 01-Intro, sl. 84].

△ **Distinzione da tenere separata** (trasversale n. 1): **JVM, JRE e JDK** non sono tre nomi per «Java installato» [fonte: 01-Intro, sl. 85]. - Lo strato infrastrutturale si chiama **JRE** (Java Runtime Environment) e ha **due livelli**: la **JVM** (Java Virtual Machine) più il JRE vero e proprio. Serve per *eseguire* applicazioni Java. - Per *sviluppare* serve «un set di strumenti più ampio, che include compilatore e altri strumenti»: il **JDK** (Java Development Kit).

Il caso limite che li separa: su una macchina con il solo JRE un JAR gira, ma non puoi compilare un sorgente. Sulla tua macchina d'esame serve il JDK (fonti.md: «Strumenti: JDK, Eclipse, JUnit, JavaFX»).

La visione — perché la VM cambia lo scenario. Se l'infrastruttura si appoggia a sua volta su una **macchina virtuale** che astrae da ciò che c'è sotto, sulla stessa VM possono convivere «DIVERSE INFRASTRUTTURE [...] in grado di interagire fra loro» [fonte: 01-Intro, sl. 80]. Java, Scala e Kotlin girano tutte sulla JVM, e per questo «componenti Java, Scala, Kotlin sono interoperabili» [fonte: 01-Intro, sl. 98]. Il riassunto del docente [fonte: 01-Intro, sl. 81]: cambia il processo di *costruzione*, perché non si genera più un programma per un processore e un SO ma «qualcosa di massimamente riusabile», e cambia il processo di *esecuzione*, perché si fa eseguire all'infrastruttura un prodotto «che non sarebbe eseguibile dal sistema operativo stand alone». Il prezzo è un «(piccolo) prezzo per questo passo extra», ma «i vantaggi sono molto superiori al costo».

La tipica obiezione, cioè «così non ho un vero eseguibile!», e la risposta [fonte: 01-Intro, sl. 94]:

«Una entità è “eseguibile” (apribile) solo se è presente lo strato software che la interpreta, non perché il suo nome finisce per “EXE”». Un eseguibile Windows non funziona in macOS, un file Word non si apre senza Word: «un JAR di Java è dunque “eseguibile” quanto un EXE di C#: ciò che conta è che sia presente l'infrastruttura di supporto».

Anche il doppio clic non fa eccezione: con la giusta configurazione del JRE i JAR si aprono così, e gli «EXE di .NET» con i classici eseguibili Windows «hanno in comune solo la desinenza», tanto che su un Windows «puro» non girano [fonte: 01-Intro, sl. 95].

.NET come termine di confronto [fonte: 01-Intro, sl. 86-91, 99]: l'articolazione è simile, con il **CLR** (Common Language Runtime) al posto della JVM. La differenza di fondo è che .NET supporta molti linguaggi, «ma non Java», ed è nativo su Windows e installabile a parte su Linux e Mac. Le slide sul csc e sul PATH sono dettagli operativi di .NET, non materia di questo corso.

7. Il kit non è solo il compilatore, e il deployment

Cosa afferma. «A differenza di C e C++, però, qui il compilatore è solo uno degli strumenti» [fonte: 01-Intro, sl. 100]. Il kit include strumenti per:

- produrre documentazione sempre aggiornata → «estrazione automatizzata di MANUALI HTML»;
- supportare il collaudo sistematico → «esecuzione automatizzata di SUITE DI TEST»;
- progettare la distribuzione, eventualmente con firma digitale per garantirne l'autenticità.

La visione — il deployment. La presentazione si chiude aprendo il modulo 03. «Deployment = distribuzione di un software», che «non può essere lasciata al “fai da te”» perché «10 installazioni sono una cosa, 1000 o 10000 sono tutt'altra faccenda!» [fonte: 01-Intro, sl. 101]. È la scalabilità del §1 applicata alla distribuzione: supportare N piattaforme «non può significare dover distribuire N eseguibili diversi». L'ideale è **un solo file per tutte le piattaforme**, cioè «un archivio di componenti compattati, simile a uno ZIP», che per Java è il **JAR** (Java ARchive). Il cerchio si chiude con il §6: un solo file basta *perché* l'eseguibile non è autocontenuto e la parte dipendente dalla piattaforma sta nel JRE.

8. L'automazione e l'AI

Il docente prende posizione [fonte: 01-Intro, sl. 32–34]. Da anni esistono strumenti che generano lo scheletro del codice dal modello del sistema, lasciando al progettista solo la *business logic*, e oggi gli strumenti di AI generano intere parti di codice. Sono utili per i compiti ripetitivi, «MA occhio al risultato: i generatori di testo sono progettati più per produrre testo gradevole che testo preciso — serve sempre una mente lucida e competente per valutare ciò che viene prodotto!». La conclusione: «saremo sempre più direttori d'orchestra, non «programmatori» — servirà sempre maggior competenza, non meno». Già alla slide 3: «usare anche l'AI ma dominandola».

Per te, che studi con Claude, la frase ha una conseguenza pratica: all'esame non c'è alcuno strumento che scriva il codice, e la competenza nel valutarlo si costruisce solo scrivendolo. È la stessa ragione per cui l'unità di verifica di FI2 è l'esercizio **a freddo**.

Casi limite

- **Il JAR che «non parte».** Se sulla macchina manca l'infrastruttura, il JAR non si apre. Non è un difetto del file: è la definizione relativa di «eseguibile» della sl. 94.
- **L'EXE di .NET su Windows senza .NET.** È lo stesso caso visto dall'altro lato: la desinenza non garantisce l'esecuzione (sl. 95).
- **Il test che non può fallire.** Un collaudo scritto sulle stesse assunzioni del progetto non intercetta un errore di modello: è il caso del bus 33 (sl. 29). Il collaudo verifica l'implementazione rispetto al progetto, non il progetto rispetto alla realtà.

- **Il costo dello strato in più.** La portabilità si paga con un passo extra a run-time (sl. 81). Il docente lo giudica piccolo, ma è un costo reale e non va dichiarato nullo.
-

Connessioni

- **Con il modulo 03 (Deployment) e l'esercitazione 03x (JAR):** la sl. 101 si interrompe esattamente sul JAR come «solo file per tutte le piattaforme». Il modulo 03 è la continuazione diretta di questa slide, non un argomento nuovo.
 - **Con S02 (JUnit) e con ogni laboratorio da LAB02 in poi:** la distinzione *failure/error* della sl. 18 è quella che leggerai nel pannello di JUnit a ogni esecuzione, fino al compito d'esame con la soglia dei 2/3 dei test.
 - **Con S00 (Installazione JDK):** è lì che la distinzione JRE/JDK della sl. 85 diventa una scelta concreta di cosa installare (la sl. 87 rimanda esplicitamente alle «slide Strumenti»).
 - **Con i moduli successivi, sulle idee «blended» della sl. 53:** le lambda expression e le chiusure tornano in 30 e 32, le funzioni come entità di prima classe in 35 (*Functional programming e Stream*), la questione dei tipi primitivi («everything is an object») in 16 (*Wrapper per tipi primitivi*). Sono agganci per titolo: il contenuto va verificato quando si apre ciascun modulo.
 - **Con IDS — Ingegneria del Software T (S2):** FI2 è testa di catena verso IDS (piano_laurea.md). Progetto separato dall'implementazione, UML, piano di collaudo, CI e build tools sono in questa presentazione solo nominati: sono il terreno che IDS presuppone.
 - **Con Diritto dell'Informatica (chiuso con 30):** il principio di *privacy by design & by default* della sl. 31 è lo stesso del GDPR che hai studiato. Qui il docente ne ricava una conseguenza ingegneristica: «non è qualcosa che si possa “aggiungere dopo”, a sistema già progettato», e questa è la stessa logica del collaudo, che si progetta «prima ancora di costruire il sistema» (sl. 13).
-

Domande di autoverifica

Da rispondere senza riaprire la lezione né le slide.

1. Perché il docente dice che «programmare» richiama una visione obsoleta? Che cosa distingue la programmazione *in-the-small* dalla progettazione *in-the-large*?
 2. Un test JUnit non passa. Quali sono i due esiti possibili, cosa li distingue, e dove cerchi l'errore in ciascun caso?
 3. Nel caso del bus 33, avrebbe aiutato un collaudo più accurato? Motiva la risposta usando la distinzione fra collaudo e progetto.
 4. Su una macchina è installato solo il JRE. Cosa puoi fare con un JAR e cosa non puoi fare con un sorgente .java? Quali sono i due livelli del JRE?
 5. Perché un JAR è «eseguibile quanto un EXE»? Descrivi la differenza fra collegamento statico e dinamico nei due «mondi».
-

Riepilogo

Perché il corso parla di infrastrutture e non solo di un linguaggio? Perché «un buon linguaggio non basta»: senza un'infrastruttura standard ognuno riscrive le proprie librerie e nascono «comunità incomunicabili». L'infrastruttura (JRE, con la JVM alla base) è l'unico strato dipendente dalla piattaforma, e rende possibile il «write once, run everywhere».

Che cos'è, esattamente, un eseguibile Java? Un archivio (JAR) di componenti in bytecode, non autocontenuto: le librerie si collegano dinamicamente a run-time. È eseguibile solo dove c'è l'infrastruttura, esattamente come un EXE è eseguibile solo dove c'è Windows, perché «eseguibile» è sempre relativo allo strato che lo interpreta.

Qual è il rapporto fra progetto, codice e collaudo? Il progetto è un artefatto distinto dal codice, precedente e disaccoppiato dall'implementazione. Il collaudo si progetta prima di costruire, fa parte del proprio lavoro e si automatizza (JUnit, CI). Non sostituisce il progetto: un modello del dominio sbagliato supera tutti i test scritti su quel modello.